

Model Selection

We will focus on methods to help with model selection in regression and classification tasks. These methods are particularly designed when the datasets are large with many predictor variables. Our primary interest is in the predictive capability of the models where the objective is to get good out of sample (test set) predictions when there are many possible predictor variables. While it is possible to get good in sample predictions by building more complicated models with more predictor variables, the concern is that there is a chance of overfitting the data. Our hope is to enable models to accurately predict outcomes on unseen data (generalization), not just on familiar training data.

1. Often, simpler models (fewer variables) are interpretable and might work better for out of sample predictions and so we might penalize models for excessive model complexity.
2. With the increase in computational power and the amount of data available, we will use the data by partitioning it into training, validation and test sets. The training set is used to estimate the model parameters. The validation set is used to do model selection while the test set is the data on which we will simply evaluate (check/report) how the model performs.

Bias-variance tradeoff

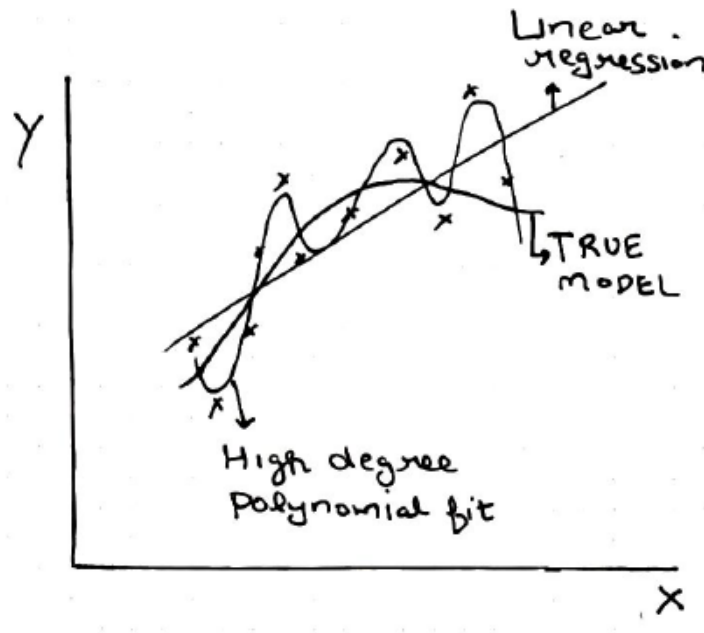


Figure 0.1: Linear regression is a simpler model (low variance but high bias), high degree polynomial regression is a complex model (high variance but low bias)

We will discuss the bias-variance tradeoff in the context of linear regression. Assume that the true model is given as:

$$y = f(\mathbf{x}) + \epsilon,$$

where ϵ is the noise term with mean 0 and variance σ^2 (not correlated with $\mathbf{x}$). Suppose we have a training set with observations $(y_i, \mathbf{x}_i)$ for $i = 1, \dots, n$ where $y_i \in \mathbb{R}$ and $\mathbf{x}_i \in \mathbb{R}^p$. We use say linear regression to develop a model $\hat{f}(\mathbf{x})$ to approximate $f(\mathbf{x})$ where the predicted value from the model is given by:

$$\hat{y} = \hat{f}(\mathbf{x}).$$

While we choose the fit of the model to minimize the sum of squared errors in the training set, we would like it to predict well on points that are not seen in the training set (generalization on the test set). We would really like to have a model, that will give a small value of

$$\mathbb{E}_{y, \mathcal{D}} \left[(y - \hat{f}(\mathbf{x}))^2 \right] \text{ (Test mean squared error)}$$

where y and $\hat{f}$ are random (here $\hat{f}$ is estimated from the training data $\mathcal{D} = \{(y_1, \mathbf{x}_1), \dots, (y_n, \mathbf{x}_n)\}$ which consist of i.i.d samples from the true data generating distribution) and $\mathbf{x}$ is fixed and given. A key equality:

$$\mathbb{E}_{y, \mathcal{D}} \left[(y - \hat{f}(\mathbf{x}))^2 \right] = \sigma^2 + \text{Variance}_{\mathcal{D}}[\hat{f}(\mathbf{x})] + \left(\mathbb{E}_{\mathcal{D}} [\hat{f}(\mathbf{x})] - f(\mathbf{x}) \right)^2.$$

The test mean squared error is decomposed into three terms:

1. Variance of the error term (irreducible): σ^2
2. Variance of the estimator: $\text{Variance}_{\mathcal{D}}[\hat{f}(\mathbf{x})]$
3. Square of the bias of the estimator: $\left(\mathbb{E}_{\mathcal{D}} [\hat{f}(\mathbf{x})] - f(\mathbf{x}) \right)^2$

To derive this equality, first assume the data set $\mathcal{D}$ is fixed ($\hat{f}$ is hence fixed):

$$\begin{aligned} \mathbb{E}_y \left[(y - \hat{f}(\mathbf{x}))^2 \right] &= \mathbb{E}_y \left[(y - f(\mathbf{x}) + f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 \right] \\ &= \mathbb{E}_y \left[(y - f(\mathbf{x}))^2 + (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 + 2(y - f(\mathbf{x}))(f(\mathbf{x}) - \hat{f}(\mathbf{x})) \right] \\ &= \mathbb{E}_y [\epsilon^2] + (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 + 2\mathbb{E}_y [\epsilon] (f(\mathbf{x}) - \hat{f}(\mathbf{x})) \\ &\quad \text{[here } y = f(\mathbf{x}) + \epsilon \text{]} \\ &= \sigma^2 + (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 \\ &\quad \text{[since } \mathbb{E}[\epsilon] = 0 \text{ and } \text{Variance}[\epsilon^2] = \sigma^2 \text{].} \end{aligned}$$

Now accounting for randomness in the data set $\mathcal{D}$:

$$\begin{aligned} \mathbb{E}_{y, \mathcal{D}} \left[(y - \hat{f}(\mathbf{x}))^2 \right] &= \sigma^2 + \mathbb{E}_{\mathcal{D}} \left[(f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 \right] \\ &= \sigma^2 + \mathbb{E}_{\mathcal{D}} \left[f(\mathbf{x})^2 + \hat{f}(\mathbf{x})^2 - 2f(\mathbf{x})\hat{f}(\mathbf{x}) \right] \\ &= \sigma^2 + f(\mathbf{x})^2 + \text{Variance}_{\mathcal{D}}[\hat{f}(\mathbf{x})] + \mathbb{E}_{\mathcal{D}} \left[\hat{f}(\mathbf{x}) \right]^2 - 2f(\mathbf{x})\mathbb{E}_{\mathcal{D}} [\hat{f}(\mathbf{x})] \\ &= \sigma^2 + \underbrace{\text{Variance}_{\mathcal{D}}[\hat{f}(\mathbf{x})]}_{\text{variance of estimator}} + \underbrace{\left(\mathbb{E}_{\mathcal{D}} [\hat{f}(\mathbf{x})] - f(\mathbf{x}) \right)^2}_{\text{square of bias of estimator}}. \end{aligned}$$

A complex model will typically have high variance for the estimator but low bias. On the other hand a simple model will have low variance of the estimator but a high bias. The key is to find the right balance between simplicity and complexity.

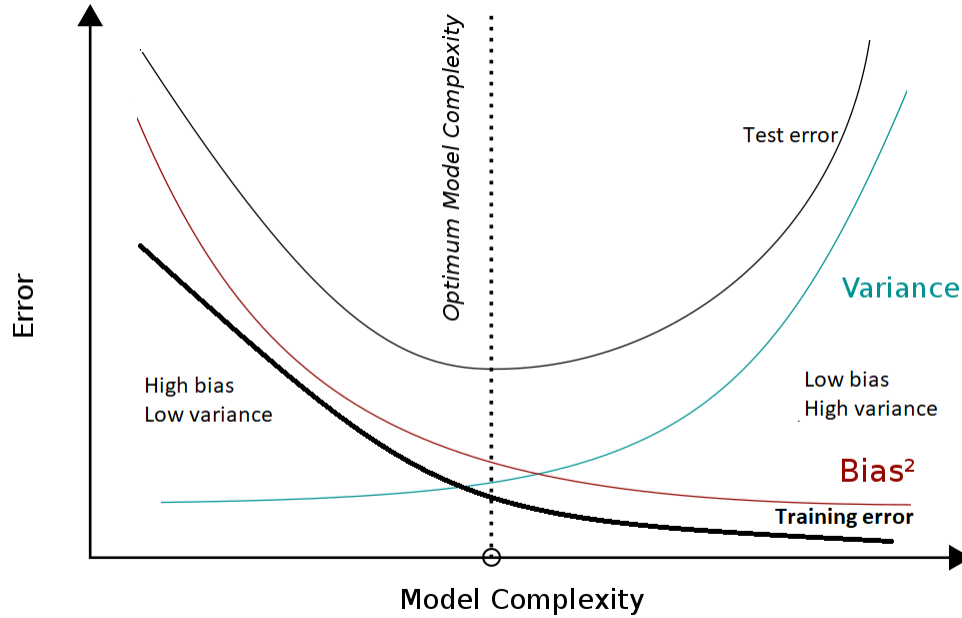


Figure 0.2: Bias-variance tradeoff

Subset selection

In classical statistical prediction, we have:

$$\text{Number of observations } n \gg \text{Number of predictors } p$$

However, there are many problems in the era of the big data where:

$$\text{Number of observations } n < \text{ or } \approx \text{Number of predictors } p$$

For example in diagnosing cancer in terms of genes, the number of genes is very large and one has to check among them to identify which genes help predict the chances of getting cancer. Note that when $n < p$ or $n \approx p$, there is a significant risk of over-fitting and there might be no longer a unique fit too. In such cases, one needs to decide which variables are important in making predictions and drop those variables that are less useful. We are also interested in identifying from a large set of variables, a much smaller subset that has the strongest effects on the response (this is fundamentally a combinatorial problem).

Key Question: Given a set of observations $(y_i, \mathbf{x}_i)$ for $i = 1 \dots, n$ with $y_i \in \mathbb{R}$, $\mathbf{x}_i \in \mathbb{R}^p$, what is the best subset of predictors for the output?

A possible algorithm is provided next.

Exhaustive Algorithm:

1. Let M_0 denote a null model with no predictors (only the intercept). This model would simply predict the mean of the training set for each observation for a linear regression model (or the fraction of ones in logistic regression).
2. For $k = 1, \dots, p$, do:
 - (a) Fit all $\binom{p}{k}$ models that contains exactly k predictors and the intercept.
 - (b) Pick the best among these models by choosing the model with the minimum sum of squared errors for linear regression or maximum log-likelihood for logistic regression. Call this model M_k .
3. Choose the best model among $M_0, M_1, \dots, M_p$ using adjusted R-squared (linear regression) or AIC (logistic regression) or cross-validation errors.

The complexity of this algorithm is exponential since we are solving $O(2^p)$ linear or logistic regression problems.

Forward stepwise selection

We discuss a computationally efficient algorithm to solve this problem in comparison to the best subset selection. However this method is not guaranteed to solve the problem to optimality. The method is based on a greedy algorithm.

1. Let M_0 denote a null model with no predictors (only the intercept). This model would simply predict the mean of the training set for each observation (or the fraction of ones in logistic regression).
2. For $k = 1, \dots, p$, do:
 - (a) Fit all models that augment the predictors in model M_{k-1} with exactly one more predictor (a total of $p - k + 1$ models are fit in step k).
 - (b) Pick the best among these models by choosing the model with the minimum sum of squared errors for linear regression or maximum log-likelihood for logistic regression. Call this model M_k .
3. Choose the best model among $M_0, M_1, \dots, M_p$ using adjusted R-squared (linear regression) or AIC (logistic regression) or cross-validation errors.

The complexity of this algorithm is much lower since we are solving $O(p^2)$ linear or logistic regression problems. On the other hand there is no guarantee that the greedy algorithm will always find the optimal solution for any fixed k .

Another similar algorithm often used is backward stepwise selection where one starts with all the predictors and then reduce one by one the least useful ones. The complexity of this algorithm is also lower than the exhaustive algorithm since we are solving only $O(p^2)$ linear or logistic regression problems. Again this does not guarantee an optimal solution.

Cross-validation

Cross-validation is a technique often used to assess (validate) the quality of the model. A model is considered good if it has a low test set error. One method of model assessment that aids in model selection is cross validation.

Validation set approach

1. Divide the data available for fitting the model randomly into two subsets (80-20, 70-30 splits): the training set and the validation set or hold-out set.
2. Use the training set to fit the model, and the validation set to predict and then obtain an estimate of the mean squared error (MSE).

The validation set method depends on the points chosen, hence different choices may lead to different estimated MSEs.

Leave out one cross-validation

Let $N = \{1, \dots, n\}$ denote the set of observations.

1. For every $i \in N$, fit the model (call it M_i) on the set $N \setminus \{i\}$ (leave out one observation).
2. Use model M_i to predict the response on observation i ; say it is $\hat{y}_i$. Compute the mean squared error:

$$\text{MSE}_i = (y_i - \hat{y}_i)^2$$

where y_i is the true response on observation i .

3. Compute the cross-validation error:

$$\text{Cross validation error} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i$$

This method has low bias, since we are fitting the model to $n - 1$ of the points (almost all the data). The output also does not change depending on the random sample we choose as in the validation set method. However it is computationally intensive: we need to fit n models.

k-fold cross-validation

This method tries to balance the drawbacks of the previous two methods.

1. Divide the data randomly into k subsets (folds) of (roughly) equal size.
2. Start with the first fold as a validation set and use the remaining $k - 1$ folds to fit the model.
3. Compute the error of the fitted model in the held-out fold.
4. Repeat steps 2 and 3 by using the second, third, fourth and so on as the hold-out fold with the remaining $k - 1$ folds to fit the model.

5. Average the error across all the k fitted models to estimate the cross-validation error.

Some of the commonly used choices for k are 5 or 10.

When we set $k = n$, this reduces to the leave out one cross validation scheme.

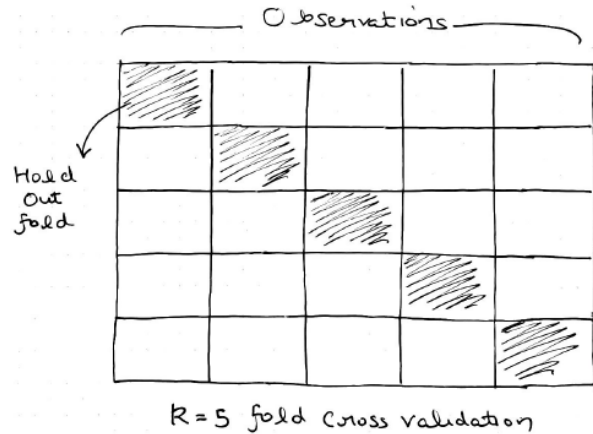


Figure 0.3: k-fold cross validation.

Once we use cross-validation to assess how different models might do on the test set, we can choose the “best” model based on smallest cross-validation error. The final chosen model is fit on the full training dataset (instead of k different models that were obtained from k -fold cross-validation). We can use this model for prediction on the test set.

Related formulation: Best subset selection problem

$$\begin{aligned} \min_{\beta_0, \beta_1, \dots, \beta_p} \quad & \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2 \\ \text{s.t.} \quad & \sum_{j=1}^p \mathbf{1}_{\beta_j \neq 0} \leq k \end{aligned}$$

where $\mathbf{1}_{\beta_j \neq 0} = 1$ if $\beta_j \neq 0$ and 0 otherwise. This can be reformulated as a quadratic integer optimization problem as follows:

$$\begin{aligned} \min_{\beta_0, \beta_1, \dots, \beta_p} \quad & \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2 \\ \text{s.t.} \quad & \sum_{j=1}^p z_j \leq k \\ & z_j \in \{0, 1\}, \quad j = 1, \dots, p \\ & -Mz_j \leq \beta_j \leq Mz_j, \quad j = 1, \dots, p, \end{aligned}$$

where M is a sufficiently large positive value.

Regularization with least absolute shrinkage and selection operator (LASSO)

In standard linear regression, the problem we solve is:

$$\min_{\beta_0, \beta_1, \dots, \beta_p} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2.$$

LASSO regularizes the linear regression model by accounting for model complexity as follows where $\lambda \geq 0$ is a parameter:

$$\min_{\beta_0, \beta_1, \dots, \beta_p} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

Here $\lambda \geq 0$ is a tuning parameter that provides a tradeoff between fitting the data (first term) with model complexity (second term in terms of non-zero values of the beta coefficients).

1. When $\lambda = 0$, LASSO reduces to standard linear regression.
2. When $\lambda \uparrow \infty$, the second term dominates and LASSO will make all the beta coefficients for the predictor variables go to zero (simpler model).

The LASSO method for linear regression was popularized by the 1996 paper “Regression Shrinkage and Selection via the Lasso” by Tibshirani. The paper had over 25000 citations as of August 2018 and over 52500 citations as of May 2023. The objective coefficient in LASSO is convex and tries to roughly promote sparsity. One of the advantages of LASSO is that since it is convex, the local optimum is the global optimum and there are efficient ways to solve the problem to optimality. However this objective function is not differentiable unlike standard linear regression.

To choose the λ values, we use a grid of possible values and compute the cross-validation error for each value of λ . We can then choose the λ with the smallest cross-validation error. You can then refit the final model using all the observations for the selected value of λ .

Summary of output from subset selection in R

Package: leaps

Function: regsubsets

1. `summary(model)$which` provides a logical matrix indicating which elements are in each model. You can also see this in the display.
2. `summary(model)$rsq` provides the R squared for each model.
3. `summary(model)$adjr2` provides the adjusted R squared for each model.
4. `summary(model)$rss` provides the residual sum of squares for each model.
5. `coef(model,k)` provides the estimated coefficients for model k.

Summary of output from LASSO in R

Package: glmnet

Function: glmnet

1. `model$a0` - This provides the intercept estimate sequence of length of the lambda sequence.
2. `model$beta` - This provides the matrix of coefficient estimates where `model$beta[,j]` gives the coefficient estimates for the jth values of the lambda sequence.
3. `model$lambda` - Lambda sequence
4. `model$df` - Number of nonzeros coefficients for each value of lambda.
5. `model$dev.ratio` - The fraction of (null) deviance explained. For linear regression problem it is just the R-squared value. The deviance is defined to be $2(\text{loglike_sat} - \text{loglike})$, where `loglike_sat` is the log-likelihood for the saturated model (a model with a free parameter per observation) and `loglike` is the log-likelihood of the model. Then `dev.ratio` = $1 - \text{dev}/\text{nulldev}$ where `nulldev` is defined to be $2(\text{loglike_sat} - \text{loglike}(\text{Null}))$ where the Null model refers to the intercept model.

For linear regression, glmnet solves the problem:

$$\min_{\beta_0, \beta_1, \dots, \beta_p} \frac{1}{2n} \underbrace{\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2}_{\text{sum of squared errors or sum of squared residuals}} + \lambda \sum_{j=1}^p |\beta_j|$$

and for logistic regression, glmnet solves the problem:

$$\min_{\beta_0, \beta_1, \dots, \beta_p} -\frac{1}{n} \underbrace{\left(\sum_{i=1}^n y_i (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) - \sum_{i=1}^n \log(1 + e^{\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}}) \right)}_{\text{log likelihood}} + \lambda \sum_{j=1}^p |\beta_j|.$$

L_0 (Cardinality)	L_1 (LASSO)	L_2 (Ridge)
$\ \beta\ _0 = \mathbf{1}_{\beta_1 \neq 0} + \dots + \mathbf{1}_{\beta_p \neq 0}$	$\ \beta\ _1 = \beta_1 + \dots + \beta_p $	$\ \beta\ _2^2 = \beta_1^2 + \dots + \beta_p^2$
Nonconvex	Convex	Convex
Discontinuous	Non-differentiable	Differentiable
Computationally hard	Computationally easy	Computationally very easy
Exact penalization that counts number of nonzero predictors	Tends to encourage sparse solutions	Encourages no term to be very large in magnitude

Table 0.1: Regularizers

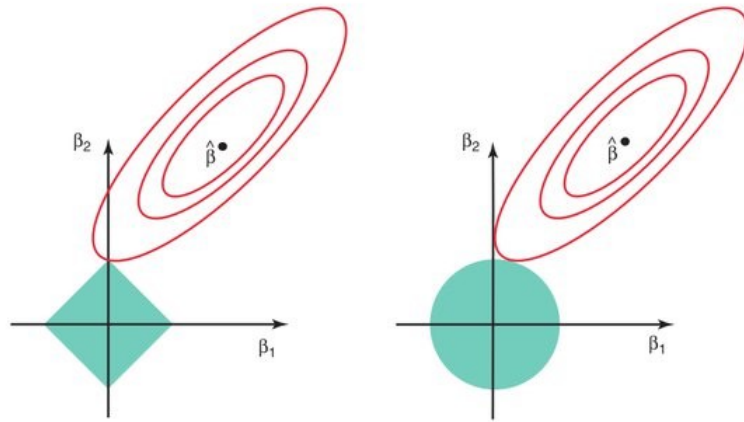


FIGURE 6.7. Contours of the error and constraint functions for the lasso (left) and ridge regression (right). The solid blue areas are the constraint regions, $|\beta_1| + |\beta_2| \leq s$ and $\beta_1^2 + \beta_2^2 \leq s$, while the red ellipses are the contours of the RSS.

Figure 0.4: Source: Textbook - Introduction to Statistical Learning